

Project Profile — MiniMax H3 Local Multimodal Video Studio on AMD Radeon

AMD AI DevMaster Hackathon 2026 · Track 1: Development of Multimodal Content Creation Tools

1. Project Background

Generative video models are becoming central to content creation — advertising, short-form social video, game cinematics, e-learning and IP asset pipelines. MiniMax H3 is an **open-weights, omni-modal** model that turns text and/or images into video with **synchronized audio**, covering text-to-video (T2V), image-to-video (I2V) and reference-to-video (R2V) in one architecture.

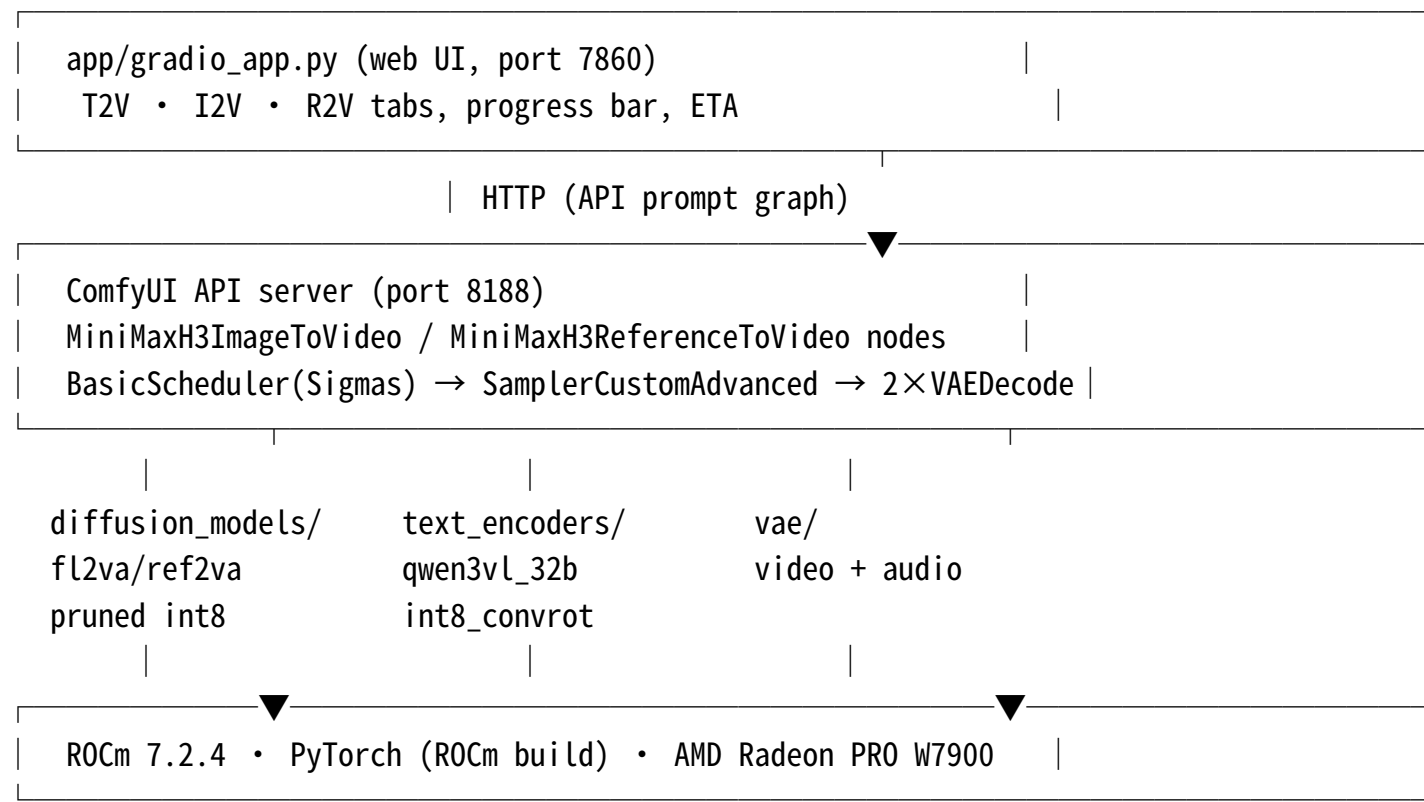
However, the open-weights release is enormous (bf16 models exceed 66 GB for the diffusion backbone alone plus a 51 GB 32B text encoder), and most inference frameworks assume NVIDIA hardware. This project adapts MiniMax H3 to run **entirely on an AMD Radeon GPU via ROCm**, using a carefully chosen **pruned + int8** weight set and ComfyUI's native nodes, and packages it as a simple local web studio.

2. Target Users & Application Scenarios

User group	Scenario	Value
Individual creators	Short-form video, social clips with synchronized sound	Free local generation, no cloud fees
Agencies / advertisers	Product ads, storyboards, reference-style character reuse (R2V)	Private data stays on-device
E-learning teams	Explainers, animated diagrams, localized voiceover audio	Audio generated in-loop, no TTS pipeline
IP / game studios	Cinematics, style-locked shots (R2V identity lock)	Rapid iteration on reference frames
Privacy-sensitive orgs	Medical / financial / legal visualization	100% local inference, no uploads

The tool is deliberately a **single-machine, single-GPU** studio: install once, start two processes, and generate.

3. System Architecture



- **Client** (`comfy_client.py`): builds ComfyUI API prompt graphs for the three modes, uploads keyframe/reference images, polls history, returns the mp4.
- **Server** (ComfyUI): runs MiniMax H3's native nodes; the omni-modal sampler decodes **video latents and audio latents in the same sampling loop**, then the two VAEs render frames + stereo audio into a single mp4.
- **UI** (`gradio_app.py`): zero-setup web studio.

4. Model & Algorithm Introduction

4.1 MiniMax H3

MiniMax H3 is an omni-modal generative model that produces video and audio from text/image conditioning. Key traits:

- **Unified video + audio generation** — one sampling pass renders both, so lip-sync/ambience/score are naturally aligned (no post-hoc dubbing).
- **T2V / I2V / R2V modes** — text-only, keyframe-constrained, or identity/style locked from reference images (`<Picture 1>` reference syntax).

- **Length grid** — frames snap to a 17k+5 grid at 24 fps (~5 s = 124 frames; trained range roughly 124–362).

4.2 Adaptation choices for AMD Radeon / ROCm

Choice	Why
pruned int8 diffusion (fl2va/ref2va, ~21 GB each)	bf16 is 66 GB — would not fit 48 GB VRAM alongside the text encoder; pruned set removes training-only weights
int8 Qwen3-VL-32B text encoder (~27 GB)	int8 (convrot) runs natively on ROCm; nvfp4-awq would need FP4 emulation on RDNA
fp16 video VAE + fp32 audio VAE	small, full precision where it matters
ComfyUI native nodes	no custom CUDA kernels; the same nodes run on ROCm PyTorch
20 steps · res_multistep · simple	Comfy-Org reference config, good quality/speed balance

5. Adaptation Description for AMD Radeon GPU / ROCm

The entire inference stack runs on AMD hardware:

- **ROCm 7.2.4** runtime with **gfx1100** (RDNA3) support — our AMD Radeon PRO W7900 is detected natively (rocm-smi: DID 0x744b, RDNA3).
- **PyTorch ROCm build** (torch 2.10.0+rocm7.2.4) drives ComfyUI's HIP kernels — no CUDA translation layers, no NVIDIA-only code paths.
- **Measured end-to-end** on the W7900 (48 GB, 864 GB/s):

Run	Resolution	Duration	Wall time
T2V cold start	864x480	~5.2 s	10:42 (model load included)
T2V warm	608x352	~2.3 s	75 s
T2V warm	864x480	~2.3 s	170 s
I2V warm	864x480	~5.2 s	~10 min (keyframe-conditioned)

Peak VRAM ~27 GB (UNet) — comfortably inside 48 GB even with the 27 GB text encoder resident.

Honest performance disclosure: image/reference-conditioned modes (I2V/R2V) are the slowest — conditioning tokens ride through

every sampling step. A ~5 s I2V clip at 864x480 takes ~9-10 min on this card (553–617 s measured). We report this openly rather than only showing best-case T2V numbers.

6. Innovation Highlights

1. **Full open-weights omni-modal studio on a single AMD GPU** — video and audio from one sampling loop, no cloud calls, no external TTS.
2. **Pruned + int8 quantization engineering** — the practical recipe (which files to pick from the official repack, why each) that makes a 190 GB+ bf16 release fit a 48 GB card.
3. **Three modes in one API client** — T2V / I2V / R2V share one prompt-graph builder; reference images ride the sampling loop for identity lock.
4. **ROCm-first setup** — PyTorch ROCm + ComfyUI native nodes only; reproducible on any RDNA2/RDNA3 card with ≥ 24 GB.

7. Reproducibility

1. ComfyUI

```
git clone https://github.com/comfyanonymous/ComfyUI.git /opt/ComfyUI
/opt/venv/bin/pip install -r /opt/ComfyUI/requirements.txt
```

2. models (HF mirror, ~75 GB)

see README §Quick start for exact hf download commands

3. ComfyUI server

```
cd /opt/ComfyUI && /opt/venv/bin/python main.py --listen 0.0.0.0 --port 8188
```

4. web studio

```
cd minimax-h3-tool && /opt/venv/bin/python app/gradio_app.py --port 7860
```

See [README.md](#) for the full dependency list, and [benchmarks/results.md](#) for performance reproduction.

8. Demo Video

[assets/demo.mp4](#) — 2:34 @ 1080p, composed of clips generated by this pipeline:

1. **Opening:** an AMD ROCm intro generated by the model (video + native audio)
2. **Title:** project name card
3. **Showcase:** generated content highlights

4. **Live operation:** real run from the web UI on the AMD Radeon PRO W7900

All opening shots were produced by the same pipeline in this repo. 9 additional generated clips (each with embedded stereo audio) ship in [assets/showcase/](#).

9. Legal & Data

- Model license: **minimax-h3-community-license-agreement** (non-commercial community license).
- All generation is local; no user data leaves the machine.